4. Bölüm

C++ Programlama Diline Giriş

4.1 En Basit C++ Programı

C programlama dili, her programcının ihtiyacını görecek genel amaçlı, emreden paradigmalı, yapısal programlama dilidir. Kısa ve öz talimatlar birbiri ardına verilerek yapılan programlamaya emereden paradigma (imperative programming) adı verilir. Yapısal programlamanın çerçevesi 2.7 başlığı altında anlatılmıştır.

Yapısal programlamada, programın başladığı yeri belirten bir **ana fonksiyon** (main function) tanımlanır. C dilinde bu fonksiyon `main()` fonksiyonudur. C++ dili, C programlama diline, nesne yönelimli programlama mantığını kodlayabilmesi için **eklenti yapılarak geliştirilmiştir**. C++ dilinde de ana fonksiyon, yine `main()` fonksiyonudur. Yani içinde `main()` fonksiyonu yazılmamış bir C++ programı çalışmaz. C++ dilinde de talimatlar noktalı virgül karakteri ile bittiğinden **gelişigüzel** (free format) yazılır. Aşağıda en basit C++ programı verilmiştir.

```
1 int main()
2 {
3     return 0;
4 }
```

Kodu şu şekilde açıklayabiliriz;

- Birinci satırda, yapısal programlamanın ana fonksiyonu olan (main function) tanımlanmıştır. Başındaki `int`, fonksiyonun bir tam sayı (integer) geri döndüreceğini belirtir. Program çalışmaya ana fonksiyondan başlar ve bittiğinde işletim sistemine geri döner.
- C dilinde her fonksiyon ikinci satırdaki gibi süslü parantez karakteri ile başlayan ve dördüncü satırdaki gibi biten bir kod bloğuna (code block) sahiptir. Her fonksiyonda olan bu blok, fonksiyon bloğu (function block) olarak da adlandırılır.
- Ana fonksiyon içinde üçüncü satırda yazılan `return 0;` talimatı (statement) ana fonksiyondan sıfır geri döndürerek çıkıldığını söyleyen talimattır. Sıfır dışında bir değerin işletim sistemine gönderilmesi, programın hata ile sonlandığı olarak kabul edilir. Eğer programdan çıkışta işletim sistemine `11` göndermek istersek bu talimatı `return 11;` olarak yazmalıyız. Ana fonksiyonda varsayılan olarak, hiçbir hata ile karşılaşılmadığı için, işletim sistemine `0` gönderilir. Bu nedenle `return 0;` talimatın yazılması zorunlu değildir. Ancak başka bir değer gönderilecek ise yazılır.

4.2 Söz Dizim Kuralları

C++ dili için oluşturulacak kaynak kod programcı tarafından metin dosyasına yazılırken belli **söz dizim kurallarına** (syntax) uyar. Bunlar aşağıda başlıklar halinde verilmiştir.

§4.2.1 Kaynak Kodumuzu Oluşturacak Karakterler

1. İngiliz alfabesindeki büyük harfler:
   `A B C D E F G H I J K L M N O P Q R S T U V W X Y Z`
2. İngiliz Alfabesindeki küçük harfler:
   `a b c d e f g h i j k l m n o p q r s t u v w x y z`
3. Rakamlar:
   `0 1 2 3 4 5 6 7 8 9`
4. Özel Karakterler:
   `< > . , _ ( ) ; $ : % [ ] # ? ' & { } " ^ ! * / | - ~ +`
5. Metinde görünmeyen ancak metni biçimlendiren **beyaz boşluk** (white space) karakterleri;
   (a) Boşluk (space)
   (b) Yeni satır (new line)
   (c) Satır başı (carriage return)
   (d) Geri (backspace)
   (e) Sekme (tab)

§4.2.2 Talimatlar ve Anahtar Kelimeler

Yüksek seviyeli bir dilde yazılmış bir **talimat** (statement), işlemciye belirtilen bir eylemi gerçekleştirmesini söyleyen cümledir. Yüksek seviyeli bir dildeki tek bir talimat, birkaç emir kodunu (instruction code) temsil edebilir. Talimatlar, yapılması gerekeni kısa ve net olarak belirten mantıksal satırlardır (logical sequence,).

Anahtar kelimeler (keywords), programlamada kullanılan ve C++ derleyicisi tarafından özel anlama sahip, önceden tanımlanmış, **ayrılmış kelimeler** (reserved words) olup programı oluşturan **talimatlar** (statement) bu kelimerler kullanılarak yazılırlar[1]. Tablo 4.1'de anahtar kelimeler verilmiştir.

Anahtar kelimeler, küçük-büyük harf ayrımı yapıldığından **her zaman küçük harflerle yazılırlar**.

`alignas` (1)	`constexpr` (1)(3)	`inline`(1) (3)	`static`
`alignof` (1)	`constinit` (4)	`int`(1)	`static_assert` (1)
`and`	`const_cast`	`long`	`static_cast`
`and_eq`	`continue`	`mutable`(1)	`struct` (1)
`asm`	`co_await` (4)	`namespace`	`switch`
`atomic_cancel`	`co_return` (4)	`new`	`synchronized`
`atomic_commit`	`co_yield` (4)	`noexcept` (1)	`template`
`atomic_noexcept`	`decltype` (1)(2)	`not`	`this` (5)
`auto` (1)(3)(4)(5)	`default` (1)	`not_eq`	`thread_local` (1)
`bitand`	`delete` (1)	`nullptr` (1)	`throw` (3)(4)
`bitor`	`do`	`operator` (1)	true
`bool`	`double`	`or`	`try`
`break`	`dynamic_cast`	`or_eq`	`typedef`
`case`	`else`	`private` (4)	`typeid`
`catch`	`enum` (1)	`protected`	`typename` (3)(4)
`char`	`explicit`	`public`	`union`
`char8_t` (4)	`export` (1)(4)	`reflexpr`	`unsigned`
`char16_t` (1)	`extern` (1)	`register` (3)	`using` (1)(4)
`char32_t` (1)	`false`	`reinterpret_cast`	`virtual`
`class` (1)	`float`	`requires` (4)	`void`
`compl`	`for` (1)	`return`	`volatile`
`concept` (4)	`friend`	`short`	`wchar_t`
`const`	`goto`	`signed`	`while`
`consteval` (4)(5)	`if` (3)(5)	`sizeof` (1)	`xor`
			`xor_eq`

Tablo 4.1: Anahtar Kelimeler

Yüksek seviyeli bir dilde yazılmış bir talimat (statement), işlemciye belirtilen bir eylemi gerçekleştirmesini söyleyen cümledir. Yüksek seviyeli bir dildeki tek bir talimat, birkaç makine dili emir kodunu (instruction code) temsil edebilir. Talimatlar, yapılaması gerekeni kısa ve net olarak belirten mantıksal satırlardır (logical sequences).

Her talimat (statement), anahtar kelimeler içerecek şekilde yazılır ve **noktalı virgül karakteri ile biter**.

Talimatlar BASIC ve FORTRAN gibi dillerde genellikle satır sonunda biter. FORTRAN dilinde yedinci sütundan yazılmaya başlanır. BASIC dilinde ise her satırın başına satır numarası yazılır. Pascal ve C dillerinde böyle bir kural söz konusu değildir. Talimat herhangi bir yerde başlar ve noktalı virgül karakteri ile biler. Aradaki beyaz boşluklar ne kadar çok olursa olsun tek bir tane varmış gibi kabul edilir. Bu nedenle kod **gelişigüzel** (free format) yazılır.

Aşağıda gelişigüzel yazılan programların hepsi derlenebilir ve derleme sonucu aynı çalıştırılabilir dosyayı üretir. Beyaz boşlukları alınmış en kısa kod aşağıda verilmiştir:

```
int main() {}
```

Kod içerisinde beyaz boşluk karakterleri anahtar kelimeleri bölmediği sürece istenildiği kadar kullanılabilir:

```
    int
        main  ( ) {
```

[1]https://en.cppreference.com/w/cpp/keyword (1)C++11, (2)C++14, (3)C++17, (4)C++20 ve (5)C++23 uyarlamasında değişti veya eklendi anlamındadır.

```
        return          0;
    }
```

Fonksiyon parantezleri içine veya fonksiyon blokları içine istenildiği kadar beyaz boşluk konulabilir. Parantezler ve fonksiyon blokları hizalı olmayabilir:

```
int main (              )
    {
  }
```

Kodun güzel görünmesi şart değildir:

```
    int main(
    )
{
        return
           0;
                }
```

Ancak kodun bir başkası tarafından bakımı yapılacağı göz önüne alınarak okunaklı yazılması çok önemlidir. **En doğrusu programı aşağıdaki sitilde** yazmaktır;

- ◇ Fonksiyonların gövdesi ve bazı kontrol talimatları kod bloklarından oluşur. Kod blokları için süslü parantez kullanılır.
- ◇ Süslü parantez içindeki kodlar, okunabilirliği artırmak için genellikle **4 boşluk** veya bir **sekme** (tab) içeriden başlar. Kodu bu şekilde yazmaya **girintili** (indented) yazma adı verilir.
- ◇ Ayrıca değişkenler ve anahtar kelimeler arasında boşluk karakteri veya sekme karakteri kullanılmalıdır:

```
int main() {
    //Ana fonksiyon bloğuna ait girinti
    int sayac=0;
    if (sayac==0) { //iç blok
        //iç bloğa ait girinti...
        int sonuc=1;
        return sonuc;
    }
    if (sayac==1) // Blok başlangıcı böyle de yazılabilir
    { //iç blok
        //iç bloğa ait girinti...
        int sonuc=1;
        return sonuc;
    }
}
```

§4.2.3 Açıklamalar

Talimatlar (statement), yapılması gerekeni kısa ve net olarak belirten **mantıksal satırlardır** (logical sequence). Programcı kodu yazarken **açıklama** (comment) yapma gereği duyabilir. Bunu iki şekilde yapar;

1. Kod metninde bulunulan yerden sonra satırın sonuna kadar olan bir açıklama yapılmak istendiğinde, açıklama öncesinde iki adet bölü (`//`) karakteri kullanılır.
2. Eğer birden çok satırı içeren bir açıklama yapılacak ise açıklama (`/*`) ile (`*/`) karakterleri arasına alınır.

Aşağıda açıklamalar eklenmiş ana fonksiyon örneği yer almaktadır;

```
/*
Bu program, açıklama (comment) içeren basit bir ana fonksiyonu olan C++ programıdır.
İlhan ÖZKAN, Ocak 2025
*/
int main() // Ana Fonksiyon
{   //fonksiyon bloğu başlagıcı
    /*
    Ana Fonksiyon, programın başladığı yerdir. Ana fonksiyonu olmayan bir program çalışmaz.
    */
} //fonksiyon bloğu bitişi
```

İstediğimiz kadar açıklama yapabiliriz. Derleyici açısından bunun önemi yoktur. Çünkü derlemenin ilk aşamasında **bütün bu açıklamalar ve beyaz boşluk karakterleri metinden çıkartılır**.

4.3 Değişken Tanımlama

Her yapısal programlama dilinde olduğu gibi C dilinde de **veri yapıları** (data structure) ve **kontrol yapıları** (control structure) birbirinden ayrıdır. Dolayısıyla veriyi işleyecek kontrol yapısı kodlanmadan önce veriyi taşıyan veri yapıları tanımlanmalıdır.

Değişkenler (variable) veri yapılarının temel öğeleridir. Bu başlıkta anlatılan tam sayı ve kayan noktalı sayı değişkenleri, belleğin belli bir bölgesini işgal eder ve fiziki olarak bellek bitlerden oluştuğundan ikili sayı (binary) olarak veri tutarlar.

§4.3.1 Tam Sayı Veri Tipleri

C++ dilinde 2.4 başlığında anlatılan tam sayılar (integer), **ilkel veri tiplerinden** (primitive data type) biridir. Tam sayı veri tipleri için aşağıdaki anahtar kelimeler kullanılır;

Veri Tipi	Boyut (Byte/Bit)	Kullanım Amacı / Özellik	Mimari Notu
`char`	1 Bayt (8 Bit)	İşaretli tam sayı / Karakter	En küçük adreslenebilir birim
`short`	2 Bayt (16 Bit)	Küçük ölçekli tam sayılar	Bellek tasarrufu sağlar
`int`	2/4 Bayt (16/32 Bit)	Standart tam sayı	Sistem mimarisine göre değişir
`long`	4 Bayt (32 Bit)	Geniş tam sayı	En az 32 bit garanti edilir
`long long`	8 Bayt (64 Bit)	Çok geniş tam sayı	Modern 64-bit hesaplamalar
`wchar_t`	2/4 Bayt (16/32 Bit)	Çok dilli (wide) metinler	Uluslararası karakterler
`char16_t`	2 Bayt (16 Bit)	UTF-16 Karakterler	Taşınabilir çok dilli destek
`char32_t`	4 Bayt (32 Bit)	UTF-32 Karakterler	Tam Unicode karakter desteği
`bool`	1 Bayt (8 Bit)	Mantıksal (`true`/`false`)	Arka planda 1 ve 0
`std::size_t`	Mimari Bağımlı	Nesne boyutu / İndis	`sizeof()` işlecinin sonucudur

§4.3.2 Kayan Noktalı Sayı Veri Tipleri

C++ dilinde 2.4 başlığında anlatılan kesirli sayılar için kullanılan kayan noktalı sayı (floating point number), **ilkel veri tiplerinden** (primitive data type) biridir ve bu veri tipleri için aşağıdaki anahtar kelimeler kullanılır;

Veri Tipi	Boyut (Byte/Bit)	Hassasiyet (Precision)	Kullanım Alanı
`float`	4 Bayt (32 Bit)	Tek Hassasiyet (Single)	Grafik / Düşük hassasiyetli işler
`double`	8 Bayt (64 Bit)	Çift Hassasiyet (Double)	Bilimsel ve genel hesaplamalar

§4.3.3 Diğer Veri Tipleri

Tam sayılar 4.3.1 başlığında anlatıldığı üzere standart tiplerin (`int`, `long` vb.) boyutları platformdan platforma değişebilir. Profesyonel projelerde tam kontrol sağlamak için `<cstdint>` başlığındaki sabit genişlikli tipler kullanılmalıdır. Bunlar;

- ◇ `int8_t` / `uint8_t`: Her platformda geçerli tam 8 Bit Tam Sayı
- ◇ `int16_t` / `uint16_t`: Her platformda geçerli tam 16 Bit Tam Sayı
- ◇ `int32_t` / `uint32_t`: Her platformda geçerli tam 32 Bit Tam Sayı
- ◇ `int64_t` / `uint64_t`: Her platformda geçerli tam 64 Bit Tam Sayı

Bunların dışında ileride yeri geldikçe göreceğimiz tam sayı gösterici (`std::intptr_t`) ve işaretsiz tam sayı gösterici (`std::uintptr_t`) tipleri de vardır. Bu, düşük seviyeli sistem programlaması için çok önemlidir.

C++ programlama dilinde bir de **hiçbir değeri olmayan ve bellekte yer kaplamayan** `void` veri tipi vardır. Bu veri tipini hiçbir değer döndürmeyen fonksiyon tanımlamalarında ve dinamik bellek kullanımlarında ileride çokça göreceğiz.

§4.3.4 Değişken Tanımlama

Değişkenlerin veri tipine (data type) bağlı olarak benzersiz bir **kimlik** (identifier) verilerek **tanımlanması** (definition) gerekir.

```
/* Bu program değişken tanımlamak için oluşturulmuştur.*/
```

```
int main()  {
    char yas; /* veri tipi "char" ve kimliği "yas" olan değişken tanımlandı. -128 ila +127
     ↪  arasında değer alabilir. */
    short kat;  /* veri tipi "short" ve kimliği "kat" olan değişken tanımlandı. -32768 ila +32768
     ↪  arasında değer alabilir. */
    float kilo; /* veri tipi "float" ve kimliği "kilo" olan değişken tanımlandı. */
}
```

Yukarıda tanımlaması yapılan `yas`, `kat` ve `kilo` değişkenleri ana fonksiyon bloğu içinde tanımlandığından, tanımlanan değişkenler sadece bu blok içinde geçerlidir!

Aynı blok içinde bir değişkene verilen kimlik, bir başka değişkene verilemez!

Bildirim (declaration) derleyiciye böyle bir değişken, fonksiyon veya nesne var demektir. Bunu ileride daha detaylı göreceğiz. Şimdilik tanımlama gibi kabul edebilirsiniz. **Tanımlama**, (definition) derlemenin bağlama (link) ile birlikte sorunsuz olabilecek şekilde eksiksiz yapılan bir işlemdir.

§4.3.5 Değişken Kimliklendirme Kuralları

C++ programlama dilinde **değişkenlerin kimliklendirilmesinde** (identifier definition) dört temel kuralar vardır. Bunlar;

1. **Kimliklendirmede anahtar kelimeler kullanılamaz!**
2. **Kimliklendirme asla rakamla başlayamaz!**
3. **Kimlikler en fazla 32 karakter olmalıdır!** Daha fazlası derleyici tarafından dikkate alınmaz!
4. **Kimliklendirmede ancak İngiliz alfabesindeki Büyük ve Küçük harfler, rakamlar ile alt çizgi karakteri kullanılabilir**.

Aşağıda kimliklendirme kurallarına uygun tanımlama yapılmıştır;

```
/* Bu program değişken tanımlama örneklerini içerir. */
int main() {
    /* Tam sayı değişkenler: */
    char yas;
    short kisa_tam_sayi;
    long uzun_tam_sayi;
    /* Gerçek Sayı Değişkenler:*/
    float kilo;
    double reel_sayi;
    /* Aynı veri tipinde birden fazla değişkene aralarına virgül koyarak kimlik verilebilir. */
    int kat1,kat2,kat3;
    float olcek1,olcek2;
}
```

Bütün bu kuralların yanında değişkenlere kimlik verilirken yazılan kodun okunaklılığını artırmak için **deve yazımı** (camel case) stili kimlik verilir. Bu stilde ilk sözcük tamamıyla küçük, izleyen sözcüklerin ise baş harfi büyük yazılır. Bu kural zorunlu değildir ama koda bakım yapacak sonraki programcıların işini kolaylaştırmak için ahlaki olarak tercih edilir;

```
/* Bu program camelCase değişken kimliklendirme örneklerini içerir.*/
int main() {
    int sayac;
    int ogrenciBoyu;
    int asansorunOlduguKat;
    float asansorAgirligi;
    int ogrenciYasi;
    char ilkHarf;
    char ogrenciAdi[50];
    char ogrenciSoyadi[50];
    float ortalamaNot;
}
```

Değişkenin kapsamı (variable scope); Değişkenin bildiriminin (variable declaration) yapılabileceği, değişkene erişilebileceği (access), değişkenin üzerinde çalışılabileceği program kodu olarak tanımlanır. Değişkenin kapsamı, tanımlandığı kod bloğu (ve varsa bu blok içindeki iç bloklar) ile sınırlıdır.

§4.3.6 Veri Tipi Değiştiricileri

İşaretli bir tam sayı kullanmak yerine en anlamlı işaret bitini de sayıya dahil ederek tam sayıları **işaretsiz olarak** (unsigned) kullanabiliriz. Tam sayı veri tipleri olan `char`, `short`, `int` ve `long` veri tipleri (data type) aslında **işaretli** (signed) tam sayılardır. Yani önlerinde `signed` sıfatı olduğu varsayılır. Tam sayıları işaretsiz olarak kullanmak istememiz halinde `unsigned` sıfatını tam sayı veri tipinin önünde kullanırız.

```
/* Bu program işaretsiz tam sayı değişken tanımlamak için oluşturulmuştur.*/
int main()  {
    unsigned yas; /* veri tipi "unsigned char" ve kimliği "yas" değişkeni
    tanımlandı. yas değişkeni 0 ile 255 arasında bir değer alabilir. */
    unsigned short kat; /* veri tipi "unsigned short" ve kimliği "kat" olan değişken
     ↪  tanımlandı.kat değişkeni 0 ila 65535 arasında değer alabilen bir değişkendir. */
    unsigned long pozitifTamsayi; /* pozitifTamsayi değişkeni 0 ile 4.294.967.295 arasında değer
     ↪  alabilen bir değişkendir. */
    uint8_t ogrenciYasi; //işatersiz veri tipi kullanarak yaş tanımı.
}
```

Yukarıda 4.3 başlığında verilen temel veri tiplerini değiştiren `unsigned` gibi **veri tipi değiştiricileri** (data type modifier) vardır. Değiştiriciler için kullanılan anahtar sözcükler;

- Bir değişkenin değerinin ilk değer atamasından sonra değiştirilemeyeceğini `const` sıfatı belirtir.
- Bir veri tipinin hem pozitif hem de negatif değerleri tutabileceğini `signed` sıfatı belirtir. Örneğin 8 bitlik `char` veri tipinin en anlamlı olan 8. bitini işaret biti olarak kullanılmasını sağlar. Böylece -127 ile +128 arasındaki sayılar `signed char` olarak tanımlanabilir.
- Bir veri tipinin yalnızca negatif olmayan değerleri tutabileceğini `unsigned` sıfatı belirtir. Örneğin 8 bitlik `char` veri tipinin en anlamlı olan 8. bitini sayı olarak kullanılmasını sağlar. Böylece 0 ile 255 arasındaki sayılar `unsigned char` olarak tanımlanabilir.
- Tam sayı tipinin daha kısa bir sürümünü `short` belirtir. Yani 16 bit tam sayı tanımlamak için `short int` veri tipi kullanılabilir.
- Tam sayı tipinin daha uzun bir sürümünü `long` belirtir. Yani 32 bit tam sayı tanımlamak için `long int` veri tipi kullanılabilir.
- Bir değişkenin değerinin programcının hazırladığı mevcut kod dışında beklenmedik şekilde değişebileceğini ve derleyicinin bazı optimizasyonları yapmaması gerektiğini `volatile` sıfatı belirtir. Örnek olarak bir aygıt tarafından veya bir **kesme** (interrupt) tarafından bir değişken değiştirilebilir. Derleyiciye bir değişkenin değerinin kodu yazılmış program tarafından değil de başka şekillerde değiştirilebileceğini söyler.

```
/* Bu program işaretsiz tam sayı değişken tanımlamak için oluşturulmuştur.*/
int main() {
    unsigned char ogrenciYasi; /* veri tipi "unsigned char" ve kimliği "ogrenciYasi" olan
    değişken bildirimi. Bu değişken 0 ile 255 arasında bir değer alabilir. */
    unsigned short asansorunBulunduguKat;  /* veri tipi "unsigned short" ve kimliği
    "asansorunBulunduguKat" olan değişken bildirimi. Bu değişken, 0 ile 65.535 arasında
    değer alabilen bir değişkendir */
    unsigned long int pozitifTamsayi; /* pozitifTamsayi değişkeni 0 ile 4.294.967.295 arasında
     ↪  değer alabilen bir değişkendir. */
    volatile int deviceStatus=0; /* Programcının yazacağı kod dışında bu değişken
     ↪  değiştirilebilir! */
}
```

> Profesyonel yazılım mimarisi ve taşınabilir kod için, sabit bit sayısına sahip işaretsiz veri tiplerinin (`uint8_t`, `uint16_t`, `uint32_t`, `uint64_t`) kullanılması şiddetle tavsiye edilir. Tam sayı veri tipleri 4.3.1 başlığında bu tiplere yer verilmiştir. Bu tipler, tüm platformlarda tam olarak doğru bit sayısını garanti eder.

4.4 Değişmezler

Programcı, kodlama yaparken bazı **sabitleri** (constant) ister istemez kullanır. Sabitler iki türlü tanımlanır. Bunların biri **değişmez** (literal), diğeri ise değeri değiştirilemeyen **sabit değişkenlerdir** (const variable). Derleme öncesinde de sonrasında da kaynak koddakinin kelimesi kelimesine aynısı olan **değişmezler** (literal) kullanılır. Değişkenlere verilen **ilk değerler** (initial value) ile **katsayılar** (coefficient) en çok kullanılan değişmezlerdir.

Değişkenler tanımlanırken sahip olacağı ilk değerleri değişmezler ile verme işlemine **başlatma** (initialize) adı verilir.

Aşağıda çeşitli değişkenleri başlatma örneği verilmiştir;

```
1 /* Bu program değişkenleri değişmez (literal) ile başlatma örneklerini içerir. */
2 int main() {
3     short i,j=0;
4     char c=65;
5     float f=2.5f;
6     char harf='A'; //char harf=65;
7     bool dogruMu=false;
8 }
```

Yukarıda verilen kod şu şekilde açıklanabilir;

- 3. Satırda; `short` veri tipindeki `i` değişkeni tanımlandı ve yine `short` veri tipindeki `j` kimlikli değişkene 0 tam sayı değişmezi (integer literal) ile başlatıldı.
- 4. Satırda; `char` veri tipindeki `c` kimlikli değişkende 65 tam sayı değişmezi ile başlatılmıştır.
- 5. Satırda; `float` veri tipindeki `f` kimlikli değişkene Türkçe yazılışı `2,5` olan kayan noktalı değişmezi (`f` harfi tek incelikli olduğunu ifade eder) ile başlatıldı. Kod İngilizce yazıldığından, kodun içerisindeki gerçek sayı değişmezleri için **ondalık ayracı nokta olarak yazılır**.
- 6. Satırda; `char` veri tipindeki `harf` kimlikli değişkene `'A'` karakter değişmezi (character literal) ile başlatıldı. **Karakter değişmezler tek tırnak içinde yazılır.** Klavyeden basılan her bir harf bellekte sayı olarak tutulur. `'A'` karakterinin kodu 65, `'B'` karakterinin kodu 66, ... Aşağıda detaylı açıklama verilmiştir.
- 7. Satırda; `bool` veri tipindeki `dogruMu` kimlikli değişkene `false` bool değişmezi (bool literal) ile başlatıldı.

Karakter kodları için genellikle ASCII (American Standard Code for Information Interchange) kümesi kullanılır ve 0 ile 127 arasındaki karakterleri içeren 7 bitlik bir karakter kümesidir. ANSI (American National Standards Institute) kümesi ise, 8 bitlik karakter kümeleri için kullanılır. Bu karakter kümeleri, değişmemiş ASCII karakter kümesini içerir. ANSI karakter kümesi, ek olarak, 128 ila 255 arasında ek karakterler içerir. Bunlar Batı Avrupa özel karakterleri ile Arapça, Yunanca veya Kiril karakterleri gibi karakterlerdir.

Kodu `0` olan karakter ise **boş karakter** (NULL Character) olarak adlandırılır.

Aşağıda birçok veri tipi için değişmez (literal) örneği tablo olarak verilmiştir. Bunların birçoğu ilerleyen konularda anlatılacaktır. Bu tabloda değişmezler tanımlanırken her zaman büyük harf son ekleri (örneğin `L`

Kategori	Veri Tipi	Değişmez Örneği	Açıklama/Önek/Sonek
Tam Sayı	`int`	42	Varsayılan (onluk)
	`unsigned int`	`42u` veya `42U`	`u` veya `U` soneki
	`long`	`42l` veya `42L`	`l` veya `L` soneki
	`unsigned long`	`42ul` veya `42UL`	`ul` soneki
	`long long`	`42ll` veya `42LL`	`ll` veya `LL` (64-bit)
	İkili (binary)	`0b101010`	`0b` ön eki (C++14)
	Onaltılı (hexadecimal)	`0x2A`	`0x` ön eki
Kayan Noktalı	`double`	`3.1415`	Varsayılan (8 byte)
	`float`	`3.14f` veya `3.14F`	`f` veya `F`(4 byte)
	`long double`	`3.14l` veya `3.14L`	`l` veya `L` (12-16 byte)
Karakter	`char`	`'A'` veya `"ABC"`	Standart (UTF-8/ASCII)
	`wchar_t`	`L'A'` veya `L"ABC"`	`L` ön eki (Geniş Kartakter)
	`char8_t`	`u8"ABC"`	`u8` öneki (C++20)
	`char16_t`	`u'A'` veya `u"ABC"`	`u` ön eki (UTF-16)
	`char32_t`	`U'A'` veya `U"ABC"`	`U` ön eki (UTF-32)
Mantıksal	`bool`	`true`, `false`	Doğru veya Yanlış
Pointer	`std::nullptr_t`	`nullptr`	Boş gösterici (C++11)

Tablo 4.2: Tüm Veri Tipleri İçin Değişmez Örnekleri

veya LL) tercih edilir. Küçük l, bazı fontlarda bir rakamı ile karıştırılabildiği için modern yazım standartlarında büyük harf daha güvenlidir. Ayrıca ileride göreceğimiz, C++11 ile gelen **kullanıcı tanımlı değişmez** UDL (user defined literal) işleci (operator"") sayesinde 100.0_km gibi fiziksel birimleri koda ekleyebiliriz.

Aşağıdaki kod örneğinde programcı, sırasıyla 3, 3.14, 2.0 ve 3.14 olmak üzere dört farklı değişmez (literal) kullanılmıştır. Bu değişmezlerin her biri derleyici tarafından farklı olarak değerlendirilir ve çalışma anında (run-time) değiştirilemez.

```
/* Bu program, değişmez örneğidir. */
int main() {
    int yariCap=3;
    float daireninAlani=3.14*yariCap*yariCap;
    float daireninCevresi=2.0*3.14*yariCap;
}
```

Değişkenler tanımlanırken parantez içerişinde değişkeler verilerek de başlatılabilir. Aşağıda yeni nesil **başlatma** (initialize) örnekleri verilmiştir;

```
/* Bu program değişkenlere C++ tipi ilk değer verme ve değişmez örneklerini içerir. */
int main() {
    int tamsayi(61); /* "tamsayi"  kimlikli "int" veri tipindeki değişken tanımlandı.
    ve ilk değeri 61 değişmezi ile verildi */
    char enter(13), tab(8), buyukA('A'); /* "enter" kimlikli "char" veri tipindeki
    değişkene 13 değişmezi ile ilk değer verildi.  "tab" kimlikli "char" veri tipindeki
    değişkene 8 değişmezi ile ilk değer verildi. "buyukA" kimlikli "char" veri tipindeki
    değişkene 'A' karakter değişmezi ile ilk değer verildi. */
    double reel(2.5); /* "reel" kimlikli "double" veri tipindeki değişkene Türkçe 2,5
    değişmezi ile ilk değer verildi. Ondalık ayracı NOKTA olarak yazılır. */
    bool test(false); /* "test" kimlikli "bool" veri tipindeki değişkene
    false değişmezi ile ilk değer verildi. false yerine 0 true yerine 1 de yazılabilir.*/
}
```

4.5 Sabit Değişkenler

Değişmezlerin (literal) çokça kullanılması derleme sonrası üretilen icra edilebilir makine kodunu da büyütür. Bunun yerine çok kullanılan değişmezler bellekte bir kez yer kaplasın diye **const sıfatıyla** (const qualifier) **sabit değişkenler** (const variable) tanımlanabilir. Sabit değişkenler etik olarak büyük harfle kimliklendirilir. **Sabit değişkenlere de ilk değerlerle başlatılmalıdır (initialize)!**

```
/* Bu program, const sıfatı örneğidir. */
int main() {
    int yariCap=3;
    const float PI(3.14); // Sabit değişken tanımı!
    float daireninAlani=PI*yariCap*yariCap;
    float daireninCevresi=2.0*PI*yariCap;
}
```

Böylece aynı sabit PI sabit değişkeni birden çok yerde kullanılır ve her seferinde aynı bellek bölgesine erişildiğinden bellekten tasarruf edilmiş olunur. Tanımlama (definition) sırasında const sıfatı kullanılan değişkene, verilen ilk değer (initial value) değişmez (literal) olarak verilmelidir. **Daha sonra bu değişkene bir değer ataması olamaz!**

Bir değişkeni sabit tanımlamanın üstünlükleri aşağıda sıralanmıştır;

- **Gelişmiş Kod Okunabilirliği**: Bir değişkene const sıfatı vermek, diğer programcılara değerinin değiştirilmemesi gerektiğini belirtir; bu da kodun anlaşılmasını ve sürdürülmesini kolaylaştırır.
- **Gelişmiş Veri Tipi Güvenliği**: const kullanılması, değerlerin yanlışlıkla değiştirilmemesini sağlayabilir ve kodumuzda hata ve eksiklik olma olasılığını azaltabilir.
- **Gelişmiş Optimizasyon**: Derleyiciler, değerlerinin program yürütme sırasında değişmeyeceğini bildikleri için sabit değişkenlerini daha etkili bir şekilde optimize edebilirler. Bu, daha hızlı ve daha verimli kodla sonuçlanabilir.
- **Daha İyi Bellek Kullanımı**: Değişkenlere const sıfatı vermek, değerlerinin bir kopyasını oluşturmayı engeller; bu da bellek kullanımını azaltabilir ve performansı artırabilir.
- **Gelişmiş Uyumluluk**: Değişkenlere const sıfatı vermek, kodunuzu sabit değişkenleri kullanan diğer başlıklarla daha uyumlu hale getirebilir.
- **Gelişmiş Güvenilirlik**: const kullanarak, değerlerin beklenmedik şekilde değiştirilmemesini sağlayarak kodunuzu daha güvenilir hale getirebilir ve kodunuzdaki hata ve eksiklik riskini azaltabilirsiniz.

4.6 Sabit İfadeler

C++'ta sabit değişkenler gibi bir de **sabit ifadeler** (constant expression) vardır. Sabit ifadeler `constexpr` anahtar kelimesiyle tanımlanır. Bir değişkenin veya fonksiyonun değerinin **derleme zamanında** (compile-time) hesaplanması gerektiğini belirten bir anahtar kelimedir. C++11 ile gelmiş ve modern C++'ın en güçlü özelliklerinden biri olmuştur. Temel mantığı şudur: "Eğer bir hesaplamayı program çalışırken değil de derlenirken yapabiliyorsak, yapalım; Böylece program daha hızlı çalışır."

Sabit ifadelerin hesaplaması, derleme sırasında yapılıp sonuç doğrudan koda gömüldüğü için, çalışma anında (run-time) işlemci yorulmaz ve icra hızlanır. Değer derleme aşamasında belirlendiği için, hatalı bir durum varsa (örneğin sıfıra bölme) derleyici bunu size hemen söyler. C++'ta standart dizilerin boyutu sabit olmalıdır. `constexpr` bir değeri dizi boyutu olarak güvenle kullanabilirsiniz.

```
int x;
std::cin >> x;
const int sabit1 = x;      // DOĞRU: x çalışma anında belli olur.
// constexpr int sabit2 = x; // HATA: x derleme anında bilinmiyor!
constexpr int sabit3 = 10 * 5; // DOĞRU: Derleyici sonucu 50 olarak hesaplayıp koyar.
```

`const` ile `constexpr` anahtar kelimelerinin karşılaştırması verilmiştir;

Özellik	const	constexpr
Zamanlama	Çalışma zamanında (run-time) belirlenebilir.	Sadece derleme zamanında (compile-time) belirlenir.
Başlatma	Değişkenin değeri çalışma anında (run-time) (örnek olarak kullanıcı girişi ile) atanabilir.	Değer mutlaka derleme anında (compile-time) bilinen bir sabit olmalıdır.
Temel Amaç	Verinin değiştirilmesini engellemek (read-only).	Performans artışı ve derleme zamanı hesaplamaları.
Dizi Boyutu	Her zaman dizi boyutu olarak kullanılamaz (derleyiciye bağlı).	Güvenle dizi boyutu (array size) olarak kullanılabilir.
Fonksiyonlar	Fonksiyon dönüş değerleri her zaman sabit değildir.	Fonksiyonun sonucu derleme anında hesaplanabilir.

Tablo 4.3: C++ const ve constexpr Karşılaştırması

4.7 Ön İşlemci Makrolarıyla Değişmez Tanımı

Kod içerisinde değişmezleri (literal) tekrar tekrar yazmamanın bir yolu da `#define` **ön işlemci yönergesi** (preprocessor directive) ile **makro tanımlamaktır**. Ön işlemciler, kaynak kodları derlemeden önce derleyiciyi yönlendirerek kaynak kodlar üzerinde değişiklik yapmayı sağlar. Aşağıdaki tanımlanan `PI` makrosuyla derleyici, derlemeye geçmeden `PI` görülen yerleri `3.14` olarak değiştirmesi söylenir. Derleme bu ön işlemci işleminden sonra gerçekleşir. Makro kimlikleri sabit değişkenler gibi **büyük harfle kimliklendirilir**.

Ön işlemci makroları sonunda noktalı virgül karakteri olmaz!
Ancak, Talimatlar noktalı virgül karakteriyle biter!
Modern C++ da ön işlemci makroları sabit tanımlamak için Kullanılmaz!.

```
/* Bu program, #define ön işlemci yönergesi örneğidir. */
#define PI 3.14
/* Bu makro, derleme öncesinde PI görülen yerlere 3.14 yazılmasını sağlar. */
int main() {
    int yariCap(3);
    float daireninAlani=PI*yariCap*yariCap;
    float daireninCevresi=2.0*PI*yariCap;
}
```

Ön işlemci yönergesini yerine getiren derleyici aynı kodu aşağıdaki şekle çevirerek derleme işlemine geçer. Ayrıca ön işlemci yönergeleri yerine getirilmeden önce bütün açıklamalar ve **beyaz boşluklar** (white space) kaynak koddan kaldırılır. Bu durumda ön işlemci yönergeleri sonrasında bir üstte verilen kaynak kodumuz aşağıdaki hale döner;

```
int main(){int yariCap=3;float daireninAlani=3.14*yariCap*yariCap;float
↪ daireninCevresi=2.0*3.14*yariCap;}
```

`#define` ön işlemci yönergeleri ile çok tekrarlanan değişmezler için makro tanımlanabildiği gibi aşağıdaki

örnek verildiği şekilde kod tasnifi için de makro kullanılabilir;

```
/* Bu program, #define ön işlemci yönergesinin bir başka kullanım örneğidir. */
#define BEGIN int main() {
#define END }
#define PI 3.14
BEGIN
const char ILKHARF='A';
const float AVAGADROSAYISI=6.022e23;
float yarimRadyan=PI/2;
END
```

4.8 Yazdığımız Kod Nasıl İcra Edilir?

Derleyici tarafından işlemcinin anlayacağı makine diline çevrilen programımız aslında **yazdığımız talimatları (statement) sırasıyla çalıştırır.** C++ dilinde her talimat noktalı virgül ile bittiği 4.2.2 başlığında anlatılmıştı. Aşağıdaki örnek programı göz önüne alalım;

```
1 int main() {
2     float boy(1.80);
3     float kilo(100.0);
4     float bki;
5     float boyKare;
6     boyKare=boy*boy;
7     bki=kilo/boyKare;
8     boy=1.50;
9 }
```

Yapısal programlamada program, ana fonksiyondan başlar. Bu nedenle icra, `main()` fonksiyonu içindeki ilk talimatla başlar ve aşağıdaki sırayla icra edilir;

1. İcra edilecek ilk talimat, `float boy(1.80);` talimatıdır. (2. Satır): `boy` Değişkenine bellek tahsis edilerek değişkene `1.80` değeri atanır. Yani `boy` değişkeni `1.80` değeriyle **başlatılır** (initialize).
2. Sonra `float kilo(100.0)` talimatı icra edilir (3. Satır): `kilo` Değişkeni `100.0` ile başlatılır.
3. Daha sonra `float bki;` talimatı icra edilir (4. Satır): tanımlanan `bki` değişkenine sadece bellek tahsis edilir ve değeri belli değildir. Bazı derleyiciler değerleri sıfır olarak varsayar.
4. Daha sonra `float boyKare;` talimatı icra edilir (5. Satır): Tanımlanan `boyKare` değişkenine sadece bellek tahsis edilir ve değeri belli değildir.
5. Daha sonra `boyKare=boy*boy;` icra edilir (6. Satır): Önceden tanımlanmış `boyKare` değişkenine `boy` değişkeni kendisiyle çaarpılarak işlem sonucu atanır. `boyKare` değişkeni öncesinde tanımlanmamış olsaydı derleme hatası alacaktık.
6. Daha sonra `bki=kilo/boyKare;` talimatı icra edilir (7. Satır): `kilo` Değişkenini `boyKare` değişkenine böler ve çıkan sonucu `bki` değişkenine atar.
7. Daha sonra `boy=1.50` talimatı icra edilir (8. Satır): `boy` Değişkenine `1.50` atanarak değeri değiştirilir.

4.9 İşleçler

İşleçler (operator) matematikten bildiğimiz işleçlerdir.

$$y = 3 + 2$$

Yukarıda verilen cebirsel ifadede toplama işleci (`+`) iki tarafına bulunan argümanları toplar. Bu argümanlara **işlenen** (operand) adı verilir. Yüksek düzey dillerde de aritmetik işleçlerin yanında birçok işleç de bulunur.

> En çok kullanılan işleç, atama (`=`) işlecidir.
> **Atama işleci, sağdaki işleneni soldakine atar.**
> Bu nedenle kodlama yapılırken `2+2=x` veya `a+b=x` yazılamaz!

§4.9.1 Aritmetik İşleçler

C++ programlama dilinde, C dilinde olduğu gibi, aritmetik işleçlerin çalışma şekli **işlenenlerin** (operand) veri tipine göre değişir. Aritmetik işlemler ve atama işleci için kurallar;

1. **İşlenenlerden birinin bellekte kapladığı yer küçük ise ikisi aynı olacak şekilde bellekte daha fazla yer kaplayanına dönüştürülür** ve işlem sonra yapılır ve sonuç dönüştürülen veri tipinde üretilir. Örnek olarak `char` veri tipindeki değişken ile `long` veri tipindeki bir değişken toplanmaya çalışıldığı zaman

`char` veri tipindeki değişkenin değeri `long` veri tipine otomatik dönüştürülür. Sonuç `long` veri tipinde üretilir.

2. **İşlenenlerden biri tam sayı diğeri kayan noktalı sayı ise işlem yapılmadan önce işlenenler kayan noktalı sayıya dönüştürülür.** Örneğin `int` veri tipindeki değişken ile `float` veri tipindeki bir değişken toplanmaya çalışıldığı zaman `int` veri tipindeki değişkenin değeri `float` veri tipine otomatik dönüştürülür. Sonuç `float` veri tipinde üretilir.

İşleç	İşlenenler üzerinde yapılan işlem
`+`	Sol ve sağındaki yer alan işlenenleri toplar.
`-`	Soldaki işlenenden sağdakini çıkarır.
`*`	Sol ve sağındaki yer alan işlenenleri çarpar.
`/`	Soldaki işleneni sağdakine böler. Eğer işlenenlerin her iki de tam sayı ise sadece kalan kısmı atılır. Bu durumda sonuç yine tam sayı olarak üretilir.
`%`	Kalan (modulus) işleci, sadece tam sayıları işlenen olarak kabul eder. Soldaki işlenenin sağdakine bölümünden kalanı verir. Kalan yine tam sayıdır.

Tablo 4.4: Aritmetik İşleçler

Aritmetik işleçlere (aritmetic operator) ilişkin örneği aşağıdaki görebilirsiniz;

```
/* Bu program, aritmetik işleç örneğidir. */
int main() {
    int a = 9, b = 4, res;
    float fa=6.6, fb= 2.2, fres;
    res = a + b; // işlem sonucu res=13
    res = a - b; // işlem sonucu res=5
    res = a * b; // işlem sonucu res=36
    res = a / b; // işlem sonucu res=9/4=2 tam 1/4=2
    /* Bölme işlecinin işlenenleri tam sayı olduğunda bölme sonucundaki tam sayı olarak
     ↪ hesaplanır. Bölme işleminin tam kısmını içeren sonuç döndürlür. */
    res = a / 4.3 ;  /* res tam sayı olduğundan işlem sonucu; res=9/4.3 =9.0/4.3 =2.0930 =2 tam
     ↪ 0.0930 =2 */
    res = a % b;   // işlem sonucu res=9%4= 2 tam 1/4=1
    fres= fa / fb; // işlem sonucu res=6.6/2.2=3.00
}
```

C++ programlama dilinde tam sayı bölme işlemi işlemcinin en basit yetenekleriyle çözülür. Bu nedenle diğer programlama dillerinden ayrılır. Aşağıda buna ilişkin örnek verilmiştir.

```
/* Bu program, aritmetik işleç BÖLME örneğidir. */
int main () {
    int a = 19 / 2 ; // a = 2*9+1 = 9
    int b = 18 / 2 ; // b = 9
    int c = 255 / 4; // c = 4*63+3 = 63
    int d = 45 / 4 ; // d = 4*11+1 = 11
    double aa = 19 / 2.0 ; // aa = 9.5
    double bb = 18.0 / 2 ; // bb = 9.0
    double cc = 255 / 2.0; // cc = 127.5
    double dd = 45.0 / 4 ; // dd = 11.25
    double ee = 45 / 4 ; /* ee = 11 çünkü bölme tam sayılar arasında yapılmıştır */
}
```

§4.9.2 Tekli İşleçler

Tekli işleçler (unary operator) sadece bir işlenen üzerinde işlem yapar. Tekli demek tek bir işlenen (operand) ile işlem yapması anlamındadır.

Aşağıda tekli işleçlerin kullanımına ilişkin örnek program verilmiştir;

```
/* Bu program, tekli işleç örneğidir. */
int main() {
    int a = 5, b = 5, res;
    // Tekli eksi (unary minus)
    res = -a;
    // Önce-artırım (Pre-increment)
    res = ++a + 2;
    /* a önce 6 olur, sonra 2 ile toplanır. res=8, a=6 */
```

İşleç	İşlenen üzerinde yapılan işlem
-	Kendisinden sonra gelen işlenenin işaretini değiştirir. İşlenen pozitif ise negatif yapar, negatif ise pozitif yapar.
+	Kendisinden sonra gelen işlenenin işaretini pozitif yapar.
++	Artırım (increment) işleci, işlenenin sağında veya solunda kullanılabilir.
--	Eksiltme (decrement) işleci, işlenenin sağında veya solunda kullanılabilir.
!	Mantıksal DEĞİL işleci işlenenden önce kullanılır. İşleneninin mantıksal durumunu tersine çevirmek için kullanılır.
sizeof()	Bu işleç, kendisinden sonra gelen işlenenin bellek boyutunu bayt sayısı olarak size_t tipinde döndürür. Ayrıca bir veri tipinin (data type) bellek miktarını argüman olarak vererek öğreniriz.
&	Adres işleci (address operator), kendisinden sonra gelen işlenenin bellek adresini döndürür.

Tablo 4.5: Tekli İşleçler

```
    // Sonra-artırım (Post-increment)
    res = 2 + b++;
    /* İşlem b=5 ile yapılır (res=7), sonra b 6 olur. */
    // Değerleri sıfırlayalım
    a = 5; b = 5;
    // Önce-azaltım (Pre-decrement)
    res = --a + 2;
    /* a önce 4 olur, sonra 2 eklenir. res=6, a=4 */
    // Sonra-azaltım (Post-decrement)
    res = 2 + b--;
    /* İşlem b=5 ile yapılır (res=7), sonra b 4 olur. */
    // Mantıksal DEĞİL (Logical NOT)
    //C Dilinden gelen
    a = 1; b = 0;
    res = !a; // res = 0 (false)
    res = !b; // res = 1 (true)
    //C++ yöntemiyle
    bool ba=true, bb=false, bres;
    bres = !ba; // res = false
    bres = !bb; // res = true
    // sizeof işleci
    res = sizeof a; //4
    res = sizeof(char); //1
}
```

§4.9.3 İlişkisel İşleçler

İlişkisel işleçler (relational operator), işlenenlerin arasındaki ilişkiyi verirler. C++ dilinde, doğru (true) ya da yanlış (false) olabilen **bool veri tipinde değeri** döndürürler.

İşleç	İşlenenler üzerinde yapılan işlem
==	Eşit mi? İşleci: İki işlenenin değeri birbirine eşit ise true, değilse false verir
!=	Farklı mı? İşleci: İki işlenenin değeri birbirinden farklı ise true, değilse false verir
>	Büyük mü? İşleci: soldaki işlenenin değeri soldakinden fazla ise true, değilse false verir
<	Küçük mü? İşleci: soldaki işlenenin değeri soldakinden az ise true, değilse false verir
>=	Büyük veya eşit mi? İşleci: soldaki işlenenin değeri soldakinden fazla ya da iki işlenen eşit ise true, değilse false verir
<=	Küçük veya eşit mi? İşleci: soldaki işlenenin değeri soldakinden az ya da iki işlenen eşit ise true, değilse false verir
<=>	C++20 ile hayatımıza girmiştir. Şekilden ötürü **uzay gemisi** (spaceship operator) işleci olarak adlandırılır. Bu operatör tek bir işlemle küçüktür, eşittir veya büyüktür durumunu döndürür. İleride anlatılacaktır.

Tablo 4.6: İlişkisel İşleçler

Aşağıda verilen ilişkisel işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
/* Bu program, ilişkisel işleç örneğidir. */
int main() {
    int a = 5, b = 6;
    bool res;
    res = a<b;  // işlem sonucu res=true
    res = a<=b; // işlem sonucu res=true
    res = a>b;  // işlem sonucu res=false
    res = a>=b; // işlem sonucu res=false
    res = a==b; // işlem sonucu res=false
    res = a!=b; // işlem sonucu res=true
}
```

§4.9.4 Bit Düzeyi İşleçler

Bit düzeyi işleçler (bitwise operator) özellikle ikilik tabandaki **tam sayılarda karşılıklı bitler üzerinde yapılan işlemlerdir**. Aslında bu işlemler, işlemci üzerinde çarpma, toplama, bölme ve çıkarma işlemleri için kullanılır. Aşağıda tam sayılar üzerinde yapılan bit düzeyi işlemler gösterilmiştir;

```
/* Bu program, bit düzeyi işleç örneğidir. */
int main() {
    char a = 00000110; // 6
    char b = 00000010; // 2
    char bitwise_and = a & b; // 00000010
    char bitwise_or = a | b;  // 00000110
    char bitwise_xor = a ^ b; // 00000100
    char bitwise_not = ~b;    // 00001101
    char left_shift = b << 2; // 00000100
    char right_shift = b >> 1;// 00000001
}
```

A ve B iki tam sayının karşılıklı bitleri olmak üzere bit düzeyi VE, VEYA, ÖZEL VEYA ve DEĞİL işlemlerinin doğruluk tabloları aşağıda verilmiştir;

A	B	A & B	A | B	A ^ B	~ A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Tablo 4.7: VE, VEYA, ÖZEL VEYA ve DEĞİL İşlemlerinin Doğruluk Tabloları

İşleç	İşlenenler üzerinde yapılan işlem
&	Bit düzeyi VE (AND): Tam sayının karşılıklı bitleri VE işlemine tabi tutulur.
|	Bit düzeyi VEYA (OR): Tam sayının karşılıklı bitleri VEYA işlemine tabi tutulur.
^	Bit düzeyi ÖZEL VEYA (XOR): Tam sayının karşılıklı bitleri ÖZEL VEYA işlemine tabi tutulur.
~	Bit düzeyi DEĞİL (NOT): Tekli işleç olup sonrasında gelecek sayının bitleri DEĞİL işlemine tabi tutulur.
<<	Bit düzeyi sola kaydırma: Tekli işleç olup sonrasında gelecek sayının bitlerini bir bit sola kaydırır. En anlamlı bit taşar. Taşan bu bit, işlemcinin durum kaydedicisinde taşma bayrağına taşınır.
>>	Bit düzeyi sağa kaydırma: Tekli işleç olup sonrasında gelecek sayının bitlerini bir bit sağa kaydırır. En anlamsız bit taşar. Taşan bu bit, işlemcinin durum kaydedicisinde taşma bayrağına taşınır.

Tablo 4.8: Bit Düzeyi İşleçler

§4.9.5 Mantıksal İşleçler

Mantıksal ifadeler, doğru (true) ve yanlış (false) olabilen ifadelerdir. C++ dilinde sıfırdan farklı bir tam sayı ifade doğru (`true`), sıfır olan tam sayı yanlış (`false`) olarak değerlendirilir. **Mantıksal işleçlere** (logical operator) işlenen olarak giren ifadeler öncelikle `bool` veri tipine dönüştürülür. İşlenenler A ve B olmak üzere, mantıksal işleçlerin doğruluk tablosu aşağıda verilmiştir;

Bu işleçlerin çalışma şekli doğruluk tablosundaki gibidir ancak derleyici optimizasyonu gereği, ifadenin

A	B	A && B (VE)	A | | B (VEYA)	!A (DEĞİL)
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true
false	false	false	false	true

Tablo 4.9: Mantıksal İfadelerin Doğruluk Tablosu

bir kısmı işlenmediğinden, **şartlı olarak çalışırlar**.

İşleç	İşlenenler üzerinde yapılan işlem
&&	**Şartlı** Mantıksal VE: Eğer iki işlenen sıfırdan farklı ise 1 verir. **Soldaki işlenen yanlış (false) ise sağdakine bakılmaz.**
||	**Şartlı** Mantıksal VEYA: Eğer iki işlenenden biri sıfırdan farklı ise 1 verir. **Soldaki işlenen doğru (true) ise sağdakine bakılmaz.**
!	Mantıksal DEĞİL: İşlenenden önce kullanılır. Tekli işleçlerde verilmiştir. İşlenenin mantıksal durumunu tersine çevirmek için kullanılır. Eğer işlenen sıfırdan farklı ise 0, değilse 1 verir.

Tablo 4.10: Mantıksal İşleçler

Aşağıda verilen ilişkisel ve mantıksal işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
/* Bu program, mantıksal işleç örneğidir. */
int main() {
    int a = 5, b = 6;
    bool res;
    res = a&&b; // işlem sonucu res=true
    res = a||b; // işlem sonucu res=true
    res = !a;   // işlem sonucu res=false
}
```

§4.9.6 Atama işleçleri

Atama işleçleri (assignment operator) en çok kullanılan işleçlerdir. **Atama işleçleri her zaman sağdaki değeri sola atar!** Dolayısıyla soldaki işlenen, atama yapılacak uygun veri tipinde bir değişken (variable) olmak zorundadır. Aşağıda verilen atama işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

İşleç	İşlenenler üzerinde yapılan işlem
=	Sağdaki işleneni sola atar.
+=	Soldaki işleneni sağdaki işlenen üzerine toplar ve sonucu sola atar.
-=	Soldaki işlenenden sağdaki işleneni çıkarır ve sonucu sola atar.
*=	Soldaki işleneni sağdaki işlenenle çarpar ve sonucu sola atar.
/=	Soldaki işleneni sağdaki işlenene böler ve sonucu sola atar. Eğer işlenenler tam sayı ise kesirli kısım göz ardı edilir. Bu durumda sonuç tam sayıdır.
%=	Bu işleç tam sayılarla çalışır. Soldaki işleneni sağdaki işlenene böler ve kalanı sola atar.

Tablo 4.11: Atama İşleçleri

```
/* Bu program, atama işleçleri örneğidir. */
int main() {
    int a = 10; // a değişkenine 10 değişmezi = işleciyle atanıyor.
    a += 10; // a=a+10; ile eşdeğer: a değişkenine 10 ekleniyor.
    a -= 10; // a=a-10; ile eşdeğer: a değişkeninden 10 çıkarılıyor.
    a *= 10; // a=a*10; ile eşdeğer: a değişkeni 10 kat yapılıyor.
    a /= 10; // a=a/10; ile eşdeğer: a değişkeni 1/10 kat yapılıyor.
    a %= 10; // a=a%10; ile eşdeğer: a değişkeninin 10 a kalanı bulunup ona eşitleniyor.
}
```

§4.9.7 İşleç Öncelikleri

Cebirsel ifadelerde nasıl ki önce çarpma ve bölme sonra toplama işlemleri yapılıyor. C programlama dilinde yazılan ifadelerde de işleç öncelikleri (operator precedence) vardır. İfadelerde bir işlemi öncelikli hale getirmek için parantez içerisine alınır. En içteki parantez en öncelikli olarak gerçekleştirilir.

Başka işleçler de bulunmaktadır bunlar ilerleyen bölümlerde yeri geldikçe anlatılacaktır. Aşağıda işleç-

Öncelik	İşleç Grubu	İşleçler	Gruptaki Öncelik
1	Sonek (postfix)	`() ++ --`	→
2	Tekli (unary)	`+ - ! ++ -- sizeof()`	←
3	Çarpım	`* / %`	→
4	Toplam	`+ -`	→
5	Bit Düzeyi Kaydırma	`<< >>`	→
6	İlişkisel (relational)	`< <= > >=`	→
7	Eşitlik (equality)	`== !=`	→
8	Bit düzeyi VE	`&`	→
9	Bit düzeyi ÖZEL VEYA	`^`	→
10	Bit Düzeyi VEYA	`|`	→
11	Mantıksal VE	`&&`	→
12	Mantıksal VEYA	`||`	→
13	Atama (assignment)	`= += -= *= /= %= <<= >>= &= ^= |=`	←

Tablo 4.12: İşleç Öncelikleri

lerin önceliklerine ilişkin örnek programı lütfen inceleyiniz.

```
/* Bu program, işleç öncelikleri örneğidir. */
int main() {
    int a=10, b=5;
    a= a-b+2; // Öncelik Sırası +, -, = İşlem sonunda a=7
    a= a/b+2; // Öncelik Sırası /, +, = İşlem Sonunda a=7/5+2=3
    a= 2*10-20/2+3+(12-10); // Öncelik Sırası: (-) * / - + +
    /* İfadenin çözüm sırası:
       > 2*10-20/2+3+2
       > 20-20/2+3+2
       > 20-10+3+2
       > 10+3+2
       > 13+2
       > 15 */
}
```

4.10 Kayan Noktalı Sayı Hesapları

Kayan noktalı sayıların IEEE-754 standardında ikili tabanda tutulduğu 2.4 başlığında anlatılmıştı. Kodlarken onluk tabanda değerler verdiğimiz kayan noktalı sayılar, aslında arka planda ikilik tabanda tutulur. Bu durum yazdığımız kodlarda garip davranışların ortaya çıkmasına sebep olur. Buna örnek olarak; hemen hemen her programcının yaptığı ilk hata, aşağıdaki kodun amaçlandığı gibi çalışacağını varsaymaktır;

```
float total = 0.0f;
for(float a = 0.0; a != 2; a += 0.01f)
    total += a;
```

Acemi programcı bunun `0`, `0.01`, `0.02`, `0.03`, gibi artarak, `1.97`, `1.98`, `1.99` aralığındaki her bir sayıyı toplayacağını ve 199 sonucunu, yani matematiksel olarak doğru cevabı vereceğini varsayar. Bu kodda doğru yürümeyen iki şey olur: Birincisi yazıldığı haliyle program asla sonuçlanmaz. `a` asla `2`'ye eşit olmaz ve döngü asla sonlanmaz. İkincisi ise döngü mantığını `a<2` şartını kontrol edecek şekilde yeniden yazarsak, döngü sonlanır, ancak toplam `199`'dan farklı bir şey olur. IEEE-754 uyumlu işlemlerde, genellikle bunun yerine yaklaşık `201`'e ulaşır. Bunun olmasının nedeni, kayan noktalı sayılarının onluk tabanda kendilerine atanan değerlerin, ikilik tabanda yaklaşık değerlerini temsil etmesidir. Bu duruma aşağıdaki klasik örnek verilir;

```
#include <iostream>
int main() {
    double a = 0.1;
    double b = 0.2;
    double c = 0.3;
    if(a + b == c)
        std::cout<<"Bu satır İcra Edilmez!"; //IEEE754 standardında işlem yaparsak.
    else
        std::cout<<"Kayan Noktalı Sayılar İkili Sayı Sisteminde Tutulduğundan Yaklaşık Olarak
        ↪ Hesaplanır." ;
}
```

Programcı olarak gördüğümüz şey onluk tabanda yazılmış üç sayı olsa da derleyicinin (ve altta yatan donanımın) gördüğü şey ikili sayılardır.`0.1`, `0.2` ve `0.3`, ona mükemmel bölmeyi gerektirdiğinden (ki bu onluk sayı sisteminde oldukça kolaydır), ancak ikili sayı sisteminde imkansızdır (bu sayılar, onluk sayı sistemindeki `1/3` sayısı gibi `0.333333333333333...` şeklinde kesin olmayan biçimde depolanması gerektiğindendir);

```
int main()
{
    /*64 bitlik kayan noktalı sayılar, tam sayı kısmı dahil olmak üzere 53 basamaklı hassasiyete
     ↪  sahiptir. Aşağıda ikili olarak değişmezler tanımlanmıştır.*/
    double a = 0011111110111001100110011001100110011001100110011001100110011010;
    //onluk sistemdeki 0.1 ikili sistemde kusurlu olarak saklanır
    double b = 0011111110010011001100110011001100110011001100110011001100110011010;
    //onluk sistemdeki 0.2 ikili sistemde kusurlu olarak saklanır
    double c = 0011111110100110011001100110011001100110011001100110011001100110011;
    //onluk sistemdeki 0.3 ikili sistemde kusurlu olarak saklanır
    double a + b = 0011111110100110011001100110011001100110011001100110011001100110100;
    /*c değişkeni ile a+b nin onluk sistemde 0,3'e tam olarak eşit olmadığını unutmayın! */
}
```

4.11 Düşün ve Çöz

- **Düşün**: C dilindeki anahtar kelimelerin (keywords) niçin değişken ismi olarak kullanılamayacağını dilin söz dizimi kuralları açısından değerlendiriniz.
- **Çöz**: `a = 5`, `b = 10`, `c = 15` değerleri için `a + b * c / a - b % 3` ifadesinin sonucunu işlem öncelik sırasına göre adım adım hesaplayan bir program parçacığı yazınız.
- **Düşün**: `x = x + 1` ile `x++` ifadeleri arasında performans açısından (assembly seviyesinde) bir fark var mıdır? Araştırınız.
- **Çöz**: Bit düzeyi (bitwise) işleçleri kullanarak, bir sayının 2'nin tam kuvveti olup olmadığını bulan bir C ifadesi yazınız.
- **Düşün**: Mantıksal VE (`&&`) operatöründe kısa devre (short-circuit) değerlendirmesi nedir? İlk koşul yanlış olduğunda ikincisine bakılmaması neden önemlidir?
- **Çöz**: Virgül operatörünün (`a = (b=3, b+2)`) çalışma mantığını analiz ediniz ve a ile b'nin son değerlerini tahmin ediniz.